LAB9

Data Structure

Lab9

- The deadline for lab9 submission is 7th May at 11:59 pm.
- If you have any questions, please contact TA through email. (justin043435@hanyang.ac.kr)
- Lab scores will be uploaded to the LMS next week.
- Questions about Lab 9 scores will only be accepted during next lab session.

- Code name: {student_id}_p9.c
- The code will be tested with 5 different input files.
- Each input file will be worth 20 points
- Make sure that the output format matches the provided example exactly.

Evaluation criteria

Category	Evaluation	
p9	100	
Total	100	

- *Use GCC **13.3** version.*
- *No score will be given if there is a compile error caused by difference of gcc version*

Lab 9. Binary max heap

- Design and implement a **Binary Max Heap** using an array.
- The program receives an array of arbitrary integers through the input file and constructs the binary max heap using a **BuildHeap** operation.
- After the binary max heap is built, perform additional heap operations such as **insertion, deletion, find, level-order traversal**.

Lab 9. Binary max heap

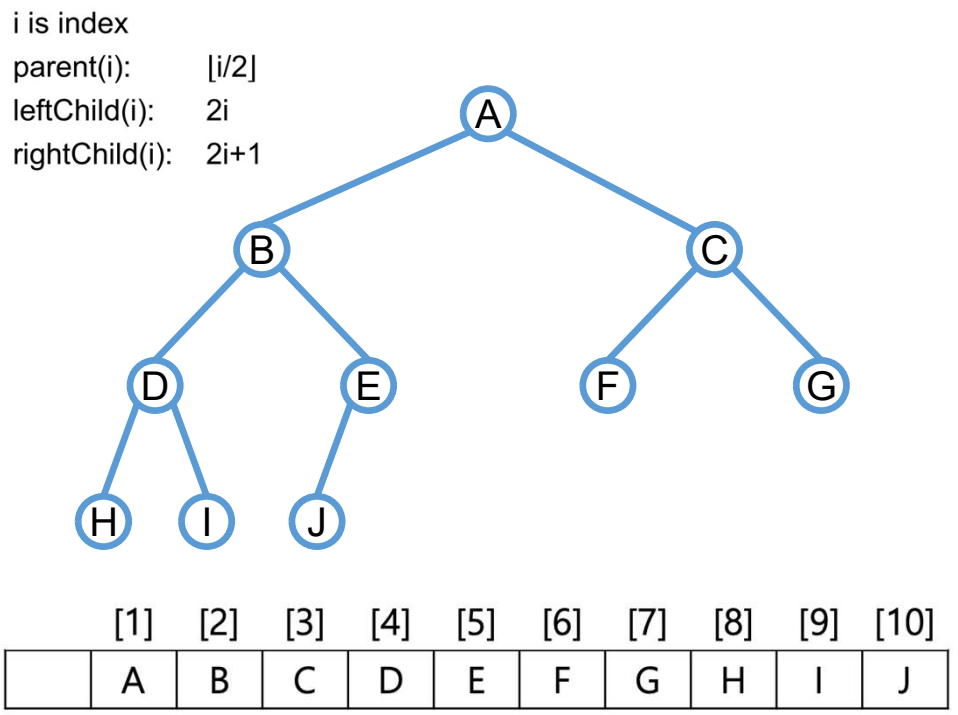
- The input file provides commands such as:
  - **n x** : Create a heap with capacity ‘x’.
  - **b x1 x2 x3 …** : Build the binary max heap from values listed after b.
  - **i x** : Insert value ‘x’ into the heap.
  - **f x** : Check if value ‘x’ exists in the heap.
  - **d** : Delete the maximum element (root).
  - **p** : Print the heap in level-order.

Lab9 – Binary max heap

- Structure

```
typedef struct HeapStruct* Heap;
struct HeapStruct {
    int Capacity;
    int Size;
    int *Element;
};
```

- Complete binary tree



Lab 9 – Binary max heap

- Structure

```
typedef struct HeapStruct* Heap;
struct HeapStruct {
    int Capacity;
    int Size;
    int *Element;
};
```

- Function

```
Heap CreateHeap(int heapSize);
void Insert(Heap heap, int value);
int Find(Heap heap, int value);
void DeleteMax(Heap heap);
void PrintHeap(Heap heap);
void BuildHeap(Heap heap, int n);
void PercDown(Heap heap, int i, int n);
void FreeHeap(Heap heap);
```

Lab 9 – Binary max heap

- **Heap CreateHeap(int heapSize)**
  - Create a heap with the size of 'heapSize'
- **void Insert(Heap heap, int value)**
  - Insert a new key to the heap.
  - If insertion is successful, print a message: "**Insert {key}**"
  - If the heap is full, print an error message: "**Insertion Error : Max Heap is full!**"
  - If the value already exists, print an error message: "**[key] is already in the heap.**"
- **int Find (Heap heap, int value)**
  - Find the key in the heap.
  - If the value exists, return 1 and print a message: "**[key] is in the heap.**"
  - Otherwise, return 0 and print an error message: "**[key] is not in the heap.**"

Lab 9 – Binary max heap

- **void DeleteMax(Heap heap)**
  - Delete the max in root node and restore the max heap property.
  - If deletion is successful, print a message: "**Max Element [key] is deleted.**"
  - If heap is empty, print an error message: "**Deletion Error : Max Heap is empty!**"
- **void BuildHeap(Heap heap, int n)**
  - Construct a max heap from the given array.
- **void PercDown(Heap heap, int i, int n)**
  - Percolate a node down to maintain the max heap property.
- **void PrintHeap(Heap heap)**
  - Print all elements in level-order.
  - If heap is empty, print an error message: "**Print Error : Max Heap is empty!**"
- **void FreeHeap(Heap heap)**
  - Free the dynamically allocated memory of the heap.

Lab 9 – Binary max heap

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char* argv[]) {
    FILE *fi = fopen(argv[1], "r");
    char cmd;
    Heap maxHeap = NULL;
    int heapSize, key;

    while ((cmd = fgetc(fi)) != EOF) {
        switch(cmd) {
            case 'n':
                fscanf(fi, "%d", &heapSize);
                maxHeap = CreateHeap(heapSize);
                break;
            case 'b': {
                if (!maxHeap) {
                    printf("BuildHeap Error : Heap not created. Use 'n x' first.\n");
                    break;
                }

                char buffer[30];
                fgets(buffer, sizeof(buffer), fi);
                char* token = strtok(buffer, " \n");
                int index = 1;

                while (token && index <= maxHeap->Capacity) {
                    int val = atoi(token);
                    maxHeap->Element[index++] = val;
                    token = strtok(NULL, " \n");
                }

                maxHeap->Size = index - 1;
                BuildHeap(maxHeap, maxHeap->Size);
                break;
            }
            case 'i':
                fscanf(fi, "%d", &key);
                Insert(maxHeap, key);
                break;
            case 'd':
                DeleteMax(maxHeap);
                break;
            case 'f':
                fscanf(fi, "%d", &key);
                if (Find(maxHeap, key))
                    printf("%d is in the heap.\n", key);
                else
                    printf("%d is not in the heap.\n", key);
                break;
            case 'p':
                PrintHeap(maxHeap);
                break;
        }
    }

    FreeHeap(maxHeap);
    fclose(fi);
    return 0;
}
```

Lab 9 – Binary max heap

- Example)

input.txt

```
n 9
b 10 50 20 15 30 60 25
p
i 10
i 70
i 65
i 101
p
d
p
f 10
f 100
d
p
d
p
d
d
d
d
d
d
d
p
```

output.txt

```
60 50 25 15 30 20 10
10 is already in the heap.
Insert 70
Insert 65
Insertion Error : Max Heap is full!
70 65 25 60 30 20 10 15 50
Max Element 70 is deleted.
65 60 25 50 30 20 10 15
10 is in the heap.
100 is not in the heap.
Max Element 65 is deleted.
60 50 25 15 30 20 10
Max Element 60 is deleted.
50 30 25 15 10 20
Max Element 50 is deleted.
Max Element 30 is deleted.
Max Element 25 is deleted.
Max Element 20 is deleted.
Max Element 15 is deleted.
Max Element 10 is deleted.
Deletion Error : Max Heap is empty!
Print Error : Max Heap is empty!
```